AI - Assisted Coding-Assignment -15.2

HTNO:2303A52228
Batch:37

Lab 15 : Backend API Development: Creating RESTful services with AI.

Task Description-1
Setup Fast API Backend.
Expected Output:
Running the server and accessing:
http://127.0.0.1:8000/
Response:
{"message": "Welcome to AI-powered FastAPI"}
Prompt:
Generate a simple FastAPI backend application in Python with one GET endpoint that returns a welcome message. The endpoint should return JSON output when accessed from the browser.
CODE AND OUTPUT :

main.py > read_root

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

PROBLEMS OUTPUT TERMINAL ··· uvicorn

```
PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2\hpc15> pip install fast
api uvicorn
Downloading annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Downloading starlette-0.52.1-py3-none-any.whl (74 kB)
Installing collected packages: annotated-doc, uvicorn, starlette, fastapi
Successfully installed annotated-doc-0.0.4 fastapi-0.135.1 starlette-0.52.1 uvico
rn-0.41.0

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2\hpc15> uvicorn main:app
 --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\ajayr\\Docume
nts\\use\\OneDrive\\Desktop\\Abhi\\3_2\\hpc15']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [6856] using StatReload
INFO:     Started server process [10892]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:50934 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:50934 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:50966 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:50966 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

127.0.0.1:8000

Pretty print ☑

```
{
  "message": "Welcome to AI-powered FastAPI"
}
```

Explanation:
The generated backend was run using VS Code. First, a Python file was created in the project folder and the required libraries were installed. Once the server started successfully, the API was tested by opening a web browser and entering the local address http://127.0.0.1:8000/. The browser displayed a JSON message.

Task Description-2
API – Create and Read Items.
Expected Output:
GET /items → []
POST /items with {"name":"Pen","price":20}
{"message":"Item created","item":{"name":"Pen","price":20}}.
Prompt:
Generate FastAPI endpoints in Python to create and retrieve items. Use a simple Python list to temporarily store products. Implement a GET endpoint to display all items and a POST endpoint to add a new item.
CODE AND OUTPUT :

main.py > ...

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

PROBLEMS OUTPUT TERMINAL ··· uvicorn

```
PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2\hpc15> uvicorn main:app
 --reload
INFO:     127.0.0.1:51189 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:51188 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51188 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51197 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51205 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:51205 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:51206 - "GET /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51227 - "GET /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51228 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:51228 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:51234 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51246 - "GET /items HTTP/1.1" 200 OK
```

```
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```


127.0.0.1:8000/items

Pretty print


```
[{"name":"Pen","price":20},{"name":"Pen","price":20},{"name":"Pen","price":20}]
```


Explanation:
The GET endpoint (/items) is used to read and display all the stored products, while the POST endpoint (/items) allows users to add new product details such as name and price. The application was executed using Uvicorn, and the endpoints were tested through the browser and the interactive API documentation provided by FastAPI.

Task Description-3
Update Existing Item.
Expected Output:
PUT /items/0 with {"name":"Pencil","price":10}
→ {"message":"Item updated","item":{"name":"Pencil","price":10}}.
Prompt:
Generate a modern event registration form using HTML, CSS, and JavaScript for a university technical fest. Include fields for Name, Email, Phone Number, Department, and Event selection. Add CSS styling for a professional look and implement JavaScript validation for email format, phone number length, and required fields with real-time error messages.
CODE AND OUTPUT:

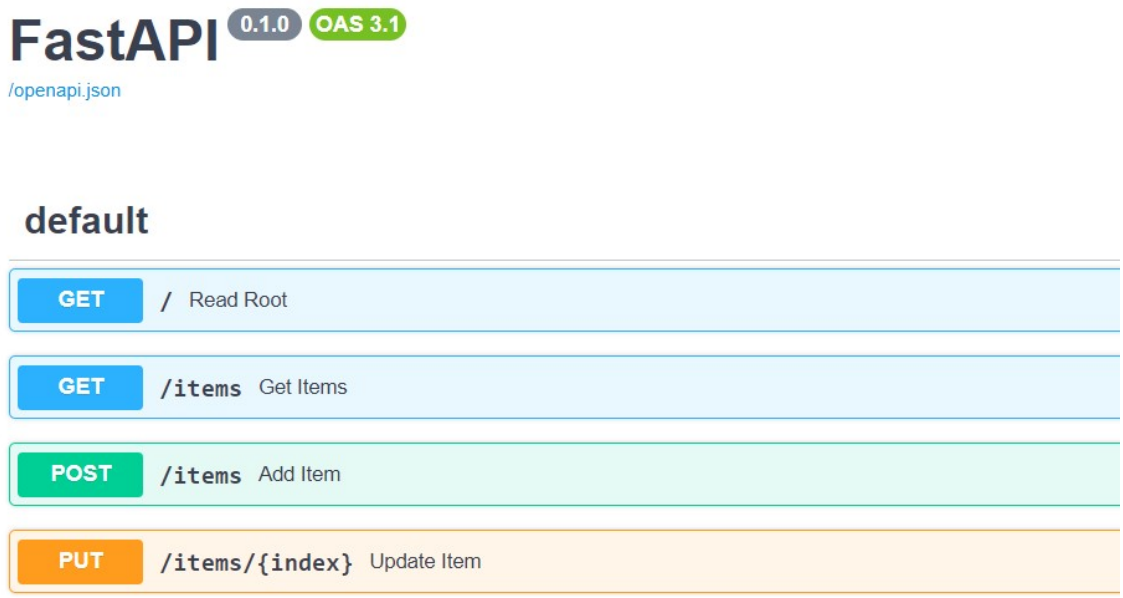
main.py > ...

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
uvicorn main:app --reload
rs\\ajayr\\Documents\\use\\OneDrive\\Desktop\\Abhi\\3_2\\hpc15']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C
 to quit)
INFO:     Started reloader process [8944] using StatReload
INFO:     Started server process [7940]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

uvicorn
uvicorn
uvicorn




```
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```



```
{
  "message": "Item updated",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```


Explanation:
AI was used to generate an endpoint in FastAPI to update an existing item using its index. The PUT request replaces the old item with new data if the index exists. If the index is not found, the API returns an error message. The API was run using Uvicorn and tested through the /docs page.

Task Description-4
Delete Item.
Expected Output:
DELETE /items/0
→ {"message":"Item removed","item":{"name":"Pencil","price":10}}.

Prompt:
Generate a FastAPI endpoint in Python to delete an item from a list using its index. If the index exists, remove the item and return a success message. If the index does not exist, return an error message.

CODE AND OUTPUT :

main.py > delete_item

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

default

GET	/	Read Root
GET	/items	Get Items
POST	/items	Add Item
PUT	/items/{index}	Update Item
DELETE	/items/{index}	Delete Item


```
{
  "message": "Item removed",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```


Explanation:
AI was used to generate a delete endpoint in FastAPI to remove an item from the list using its index value. The DELETE method checks whether the given index exists in the list. If the index is valid, the item is removed and a confirmation message is returned. If the index is not found, the API returns an error message.

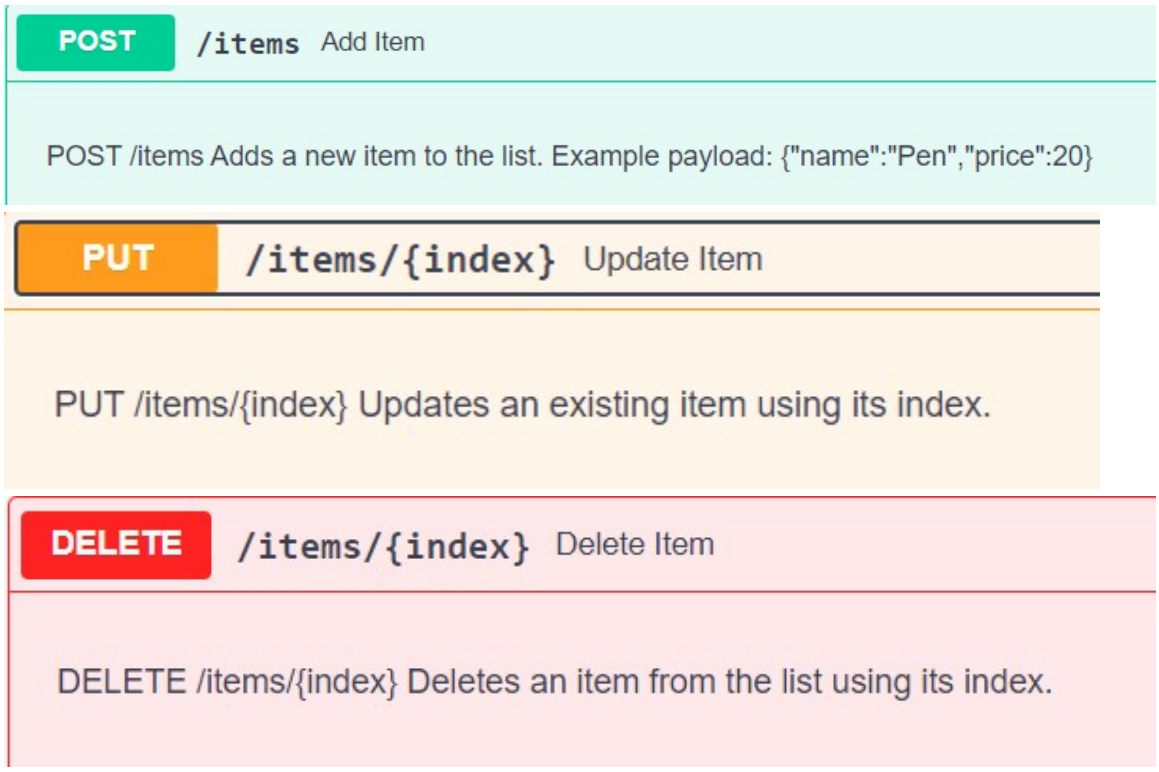
Task Description-5
Add Auto-Generated Documentation.
Expected Output:
Clear documentation for all endpoints, e.g., via /docs if using Swagger.
Students should be able to see endpoint description, methods, and sample payloads.
Prompt:
Add inline comments and docstrings to all FastAPI endpoints in a Python API application so that the endpoints are clearly documented. Ensure the documentation appears automatically in the Swagger UI provided by FastAPI.
CODE AND OUTPUT :

main.py > ...

```python
def add_item(item: dict):
    """
    POST /items
    Adds a new item to the list.
    Example payload:
    {"name":"Pen","price":20}
    """
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """
    PUT /items/{index}
    Updates an existing item using its index.
    """
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    """
    DELETE /items/{index}
    Deletes an item from the list using its index.
    """
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

GET **/items** Get Items

GET /items Returns the list of all items stored in memory.



Explanation:
AI was used to add comments and docstrings to each API endpoint created using FastAPI. These docstrings describe the purpose of each endpoint, the request method used, and the type of data expected. When the API runs using Uvicorn, FastAPI automatically generates interactive documentation through Swagger.